EDTH-LORA: Inter-Node LoRa Positioning System

Radio-Based GPS-Independent Positioning for Drone Swarms

European Defense Tech Hackathon (EDTH) · London · February 2026

48-Hour Prototype · Team of 6

Hardware: Ebyte E22-900T22S (Semtech SX1262, 868/915 MHz) | ArduPilot H7 Flight Controller
Software: Python EKF | Rust WebSocket Server | Next.js Dashboard | PlatformIO Firmware
Challenge #13 · AirQit / Mike Dewhirst

Abstract

This paper documents the design, implementation, and validation of EDTH-LORA — a radio-based, GPS-independent positioning system for drone swarms developed during the European Defense Tech Hackathon (EDTH), London, February 2026. The system addresses a critical operational gap: in GPS-contested environments such as active conflict zones, drone swarms lose all positional awareness and formation coherence. Existing resilient navigation solutions are either too expensive, too heavy, or too dependent on external infrastructure for scalable deployment.

Our approach fuses three sensing modalities: (1) LoRa sub-GHz radio modules performing inter-node ranging via Received Signal Strength Indicator (RSSI) analysis; (2) onboard Inertial Measurement Unit (IMU) sensors providing continuous dead-reckoning position estimates at 100–200 Hz; and (3) an Extended Kalman Filter (EKF) fusing both streams in real time. The architecture uses one ground-station master node and four drone-mounted nodes communicating via a Time-Division Multiple Access (TDMA) protocol. Python simulation validated EKF convergence with position error under 5 metres before hardware implementation.

Hardware integration involved the Ebyte E22-900T22S (Semtech SX1262) LoRa module, ArduPilot H7 flight controller for real-time IMU data streaming, Raspberry Pi companion computers, and a live visualisation dashboard built on Rust and Next.js. This document provides a comprehensive technical account of the full system — from the physics of RSSI propagation to the mathematics of trilateration, the EKF state equations, the TDMA protocol design, hardware integration challenges, simulation validation results, and a roadmap for future development.

Keywords: *LoRa, RSSI Ranging, Trilateration, Multilateration, Extended Kalman Filter, IMU Dead Reckoning, GPS-Denied Navigation, Drone Swarm, TDMA, Signal Propagation, Path-Loss Model, Sensor Fusion, ArduPilot, Defense Technology*

Table of Contents

1. Problem Statement and Motivation 4
2. System Requirements and Design Brief 5
3. Technical Background: RSSI, Path Loss, and Radio Propagation 6
4. System Architecture Overview 9
5. Layer 1 — LoRa Communication Protocol 10
6. Layer 2 — RSSI-to-Distance Estimation 12
7. Layer 3 — Trilateration and Multilateration 14
8. Layer 4 — IMU Dead Reckoning 17
9. Layer 5 — Extended Kalman Filter Fusion 19
10. Layer 6 — TDMA Protocol Design 22
11. Layer 7 — Visualisation Dashboard 23
12. Hardware Stack 24
13. Software Stack and Repository Structure 26
14. Simulation Design and Validation 27
15. Hardware Integration: IMU Data Logging 30
16. RSSI Calibration Methodology 32
17. 48-Hour Chronological Timeline 34
18. Team Roles and Responsibilities 35
19. Key Findings and Learnings 36
20. What Remains to Be Done 37
21. References 39

1. Problem Statement and Motivation

1.1 The Operational Context: GPS Denial in Conflict Zones

In active conflict zones — most visibly demonstrated in the ongoing war in Ukraine — GPS and GNSS signals are routinely jammed, spoofed, or denied by adversarial electronic warfare systems. This creates a fundamental crisis for autonomous drone operations: a drone swarm that relies on GPS for positional awareness becomes blind the moment jamming begins. Followers cannot maintain formation relative to the leader. Coordinated manoeuvres become impossible. The swarm degrades from an intelligent collective to a collection of disconnected, uncoordinated platforms.

The consequences extend beyond loss of formation. GPS spoofing — injecting false GPS signals — can cause drones to believe they are at a completely different location, leading to dangerous false-confidence navigation and potential swarm disintegration in adversarially controlled directions. A positioning system that treats GPS as its single source of truth has a single point of catastrophic failure.

1.2 Why Visual Positioning Fails

The natural fallback — optical and LiDAR-based positioning — fails under precisely the environmental conditions most common in contested operations: fog, clouds, mist, smoke from fires or munitions, low-light conditions, and featureless terrain. Camera-based Simultaneous Localisation and Mapping (SLAM) degrades catastrophically when visual features are absent, repeated, or obscured. These are not edge cases in a conflict environment; they are routine operating conditions.

1.3 Why Existing Resilient Solutions Are Inaccessible

Resilient alternatives exist but are commercially and operationally inaccessible for swarm-scale deployment. Ultra-Wideband (UWB) ranging arrays provide centimetre-level precision but require dense infrastructure or expensive per-node hardware. Real-Time Kinematic GPS (RTK-GPS) requires a persistent base-station connection and is vulnerable to the same jamming as standard GPS. High-grade Inertial Navigation Systems (INS) are too expensive, too heavy, and drift without correction over the timescales relevant to swarm operations. None of these solutions provides a low-cost, infrastructure-independent, scalable positioning layer for a multi-drone swarm operating in a denied environment.

1.4 The Research Question

The central question motivating this work is: can commodity radio hardware — specifically LoRa sub-GHz modules costing tens of dollars — be combined with onboard inertial sensors and probabilistic filtering to provide viable relative positioning for a drone swarm in a GPS-denied environment? The target metric is position error under 5 metres, sufficient for formation-hold operations at inter-drone separations of 10–100 metres.

2. System Requirements and Design Brief

2.1 Hackathon Challenge Specification

Challenge #13, sponsored by AirQit and Mike Dewhirst, defined the following minimum viable requirements:

- Minimum 4 nodes operating simultaneously in the swarm
- Operational range: 3 metres to 1 kilometre
- 3D relative positioning between nodes
- Master drone piloted by a human operator
- Follower drones autonomously maintain formation relative to the master
- Zero GPS dependency — the system must function with GPS completely absent

2.2 MVP Targets (48-Hour Deliverables)

- 5 Ebyte E22-900T22S modules communicating via TDMA protocol
- Ground station computing 2D position of each drone node using RSSI multilateration
- Real-time visualisation dashboard displaying node positions
- RSSI-based ranging demonstration with calibrated path-loss model
- Basic swarm command and telemetry over the same LoRa link

2.3 Stretch Goals

- Drone-to-drone ranging (full decentralised mesh positioning)
- 3D position estimation using altitude/barometer data
- Machine-learning RSSI-to-distance model for improved accuracy
- Kalman filter fusion of RSSI readings over time
- Formation flying commands with position-aware feedback
- Fingerprinting-based positioning for known indoor environments

3. Technical Background: RSSI, Path Loss, and Radio Propagation

3.1 What RSSI Is and Is Not

Received Signal Strength Indicator (RSSI) is a measurement of the power level received by a radio receiver. It is reported in decibel-milliwatts (dBm), where 0 dBm = 1 milliwatt. Higher (less negative) values indicate stronger received signal. RSSI is available on virtually all LoRa chipsets as a by-product of normal reception — no additional hardware or protocol overhead is required to obtain it.

The critical conceptual distinction for this project is that RSSI is NOT a direct measurement of distance. It is a statistical indicator of relative proximity, corrupted by a large number of physical and environmental effects. Teams that treat RSSI as a deterministic distance measurement fail; teams that treat it as a noisy constraint on a probability distribution succeed. This distinction underlies every design decision in the EDTH-LORA system.

3.2 The Log-Distance Path-Loss Model

The primary model relating RSSI to distance is the log-distance path-loss model, also known as the Friis transmission model with shadowing:

$$RSSI(d) = A - 10 \cdot n \cdot \log_{10}(d) + X_\sigma$$

A = RSSI at 1 metre (dBm) | n = path-loss exponent | $X_\sigma \sim Normal(0, \sigma^2)$ = shadowing noise

The model has three parameters that must be characterised per deployment:

- A: the RSSI value at a reference distance of 1 metre. This is calibrated by measuring RSSI with transmitter and receiver exactly 1 metre apart, averaged over many packets. Typical values: −35 to −55 dBm depending on hardware and antenna.
- n: the path-loss exponent, which governs how rapidly signal strength falls with distance. In free space, n = 2.0 (inverse-square law). In real environments: n ≈ 2.0–2.2 (open outdoor LOS), 2.4–3.0 (light indoor), 3.0–3.5+ (cluttered indoor/NLOS).
- X_σ: additive Gaussian noise representing shadowing and multipath. The standard deviation σ typically ranges from 3–5 dB in stable outdoor conditions to 6–10+ dB in dynamic indoor/rotating-antenna conditions.

Inverting the model to estimate distance from a measured RSSI value:

$$\hat{d} = 10^{((A - RSSI) / (10 \cdot n))}$$

This inversion is the key step converting radio measurements into geometric distance estimates

3.3 The RSSI Error Chain: From Signal to Position

Understanding how RSSI measurement error propagates to position error is fundamental to system design. The chain has three steps:

1. RSSI jitter (σ dB) at fixed distance: caused by antenna orientation, multipath, reflections, body shadowing. Typical indoors: σ = 5–8 dB.
2. Distance error from RSSI jitter: because the inversion is exponential (10^x), even moderate dB errors create large multiplicative distance errors.
3. Position error from distance error: trilateration amplifies distance errors geometrically, especially when anchor geometry is poor (all anchors on one side).

The distance error factor from σ dB of RSSI noise is:

```
d_est / d_true = 10^(σ / (10·n))
```

For σ = 6 dB, n = 2.2: factor = 10^(6/22) ≈ 1.87 → a '10m' reading spans 5.3–18.7m (1σ)

3.4 RSSI Reference Table: 1 m to 1000 m

The following table provides expected RSSI values and 1σ distance accuracy ranges across the full operational envelope, for two representative path-loss environments. Parameters: A = −40 dBm (calibrated at 1 m); σ = 5 dB (short range), 8 dB (mid range).

Dist (m)	RSSI n=2.2 (dBm)	Band ±6dB (n=2.2)	RSSI n=3.0 (dBm)	Band ±6dB (n=3.0)	1σ Error n=2.2	1σ Error n=3.0
1	−40.0	[−46, −34]	−40.0	[−46, −34]	×1.87 → 0.5–1.9m	×1.58 → 0.6–1.6m
2	−46.6	[−52.6, −40.6]	−49.0	[−55, −43]	×1.87 → 1.1–3.7m	×1.58 → 1.3–3.2m
5	−55.4	[−61.4, −49.4]	−61.0	[−67, −55]	×1.87 → 2.7–9.4m	×1.58 → 3.2–7.9m
10	−62.0	[−68, −56]	−70.0	[−76, −64]	×1.87 → 5.3–18.7m	×1.58 → 6.3–15.8m
20	−68.6	[−74.6, −62.6]	−79.0	[−85, −73]	×1.87 → 10.7–37m	×1.58 → 12.7–32m
50	−77.4	[−83.4, −71.4]	−91.0	[−97, −85]	×1.87 → 27–94m	×1.58 → 32–79m
100	−84.0	[−90, −78]	−100.0	[−106, −94]	×1.87 → 53–187m	×1.58 → 63–158m
200	−90.6	[−96.6, −84.6]	−109.0	[−115, −103]	Large	Very Large
500	−99.4	[−105.4, −93.4]	−121.0	[−127, −115]	IMU-primary	Near Limit
1000	−106.0	[−112, −100]	−130.0	[−136, −124]	Link Budget	At Sensitivity

Figure: RSSI vs Distance reference table. Model: A = −40 dBm, n = 2.2 (open/LOS) and n = 3.0 (indoor/cluttered). ±6 dB jitter band and 1σ distance error range shown. Replace A, n, σ with calibrated values for your environment.

3.5 Why RSSI Changes When Circling (Antenna Orientation Effects)

A critical phenomenon observed when a drone circles at approximately constant radius is large RSSI fluctuations — often ±6–15 dB — despite constant distance. This is caused by three independent physical effects operating simultaneously:

- Antenna radiation pattern: a typical whip or PCB-trace antenna has a toroidal gain pattern. As the drone rotates, the effective gain toward the receiver changes continuously, causing systematic dB-level swings correlated with yaw angle.
- Polarisation mismatch: LoRa modules typically use linearly polarised antennas. When transmitter and receiver polarisations are misaligned (e.g., one horizontal, one vertical), up to 20 dB of additional loss occurs. As the drone rotates, polarisation alignment oscillates.

- Multipath fading: indoor environments contain many surfaces that reflect and scatter radio waves. The direct-path signal and reflected signals arrive with different phase delays and combine constructively or destructively at the receiver. Moving as little as half a wavelength (~16 cm at 900 MHz) can shift a path from constructive to destructive, causing 10+ dB swings.

The practical implication: RSSI does not return to the same value when returning to the same distance from a different direction. The measurement is path-dependent and orientation-dependent, not just distance-dependent. This fundamentally motivates the use of IMU fusion and statistical filtering rather than treating RSSI as a deterministic sensor.

3.6 The Effect of Averaging: Reducing Standard Error

While the standard deviation of instantaneous RSSI (σ) is determined by the physical environment, the standard error of the mean RSSI estimate can be reduced by averaging N samples. This is the key engineering lever available without changing hardware:

$$SE_RSSI \approx \sigma / \sqrt{N}$$

SE = standard error of mean RSSI | σ = single-packet standard deviation | N = number of samples averaged

N Packets Averaged	SE (σ=3 dB)	SE (σ=5 dB)	SE (σ=8 dB)
1	3.00 dB	5.00 dB	8.00 dB
4	1.50 dB	2.50 dB	4.00 dB
9	1.00 dB	1.67 dB	2.67 dB
16	0.75 dB	1.25 dB	2.00 dB
25	0.60 dB	1.00 dB	1.60 dB
64	0.38 dB	0.63 dB	1.00 dB

Figure: Standard error of mean RSSI as a function of averaging window size. For σ = 5 dB, averaging 25 packets achieves ~1 dB SE — a 5× improvement over raw samples.

4. System Architecture Overview

The EDTH-LORA system is organised as a seven-layer stack. Each layer has a clearly defined input, output, and error model. Layers are described independently in Sections 5–11, but their interdependencies are summarised here.

Layer	Name	Input	Output	Error Source
1	LoRa Comms	Binary payload	Received packet + RSSI/SNR	Packet loss, collision
2	RSSI → Distance	RSSI (dBm)	Pseudo-range estimate $\hat{d}$ (m)	Path-loss noise σ
3	Trilateration	$\hat{d}$ from ≥3 anchors	2D/3D position estimate	RSSI error amplified
4	IMU Dead Reckoning	Acc + Gyro at 100–200 Hz	Continuous Δposition	Gyro bias drift
5	EKF Fusion	IMU + trilateration	Filtered [x, y, vx, vy]	Model mismatch
6	TDMA Protocol	Clock + node IDs	Collision-free slots	Clock drift
7	Visualisation	EKF state stream	3D live dashboard	Latency

Figure: Seven-layer EDTH-LORA system architecture with inputs, outputs, and primary error sources per layer.

The key architectural insight is that no single layer provides sufficient accuracy on its own, but their combination achieves the target. IMU provides smooth, high-rate, short-term tracking but accumulates drift. RSSI trilateration provides drift-free long-term anchoring but is noisy and low-rate. The EKF exploits the complementary frequency characteristics of these two sources — using IMU for the prediction step and RSSI for periodic updates.

The second key insight, discovered during the hackathon, is that every single LoRa packet provides a partial EKF correction — no synchronisation or time-division update schedule is required for the filter. The TDMA protocol is designed for collision avoidance and deterministic measurement scheduling, not EKF synchronisation.

5. Layer 1 — LoRa Communication Protocol

5.1 The Ebyte E22-900T22S Module

The radio hardware is the Ebyte E22-900T22S, a compact SMD module (26×16 mm) based on the Semtech SX1262 chipset operating at 868/915 MHz. Its key specifications for this application are:

Parameter	Specification
Chipset	Semtech SX1262
Frequency	868 MHz (EU) / 915 MHz (US)
TX Power	22 dBm (158 mW)
Receiver Sensitivity	−147 dBm (maximum, SF12)
Interface	UART TTL (3.3V/5V compatible)
Line-of-Sight Range	~5 km
Supply Voltage	2.7–5.5 V
TX Current	~150 mA
RX Current	~17 mA
Antenna Connector	IPEX / SMA-K
Module Size	26 × 16 mm (stamp-hole SMD)

Figure: Ebyte E22-900T22S hardware specifications relevant to the EDTH-LORA positioning system.

5.2 LoRa Modulation Parameters

LoRa (Long Range) is a proprietary chirp spread-spectrum modulation technique from Semtech. The key configurable parameters directly affect both RSSI measurement quality and system latency:

- Spreading Factor (SF, 7–12): higher SF increases link budget (sensitivity) but increases airtime quadratically. SF7 gives the lowest latency and is recommended for this application. Higher SF is only needed if range > 500m.
- Bandwidth (BW, 125/250/500 kHz): wider bandwidth reduces airtime and RSSI 'settling' effects, but reduces sensitivity. BW = 125 kHz is the standard hackathon default.
- Coding Rate (CR, 4/5 to 4/8): forward error correction. CR = 4/5 for minimum overhead.
- TX Power: fixed at 14–22 dBm across all nodes for consistent RSSI calibration. Varying TX power between nodes invalidates the path-loss model.

The critical configuration rule: all radio parameters must be identical across all nodes. Any mismatch in SF, BW, frequency, or TX power destroys the RSSI-to-distance calibration.

5.3 Packet Format

Packets are kept minimal to reduce airtime and latency. The minimum useful payload for positioning is:

```
Byte 0:    node_id       (1 byte)  — identifies which drone sent this
```

```
Bytes 1-2: rssi          (2 bytes) — received signal strength (from radio driver)
Byte 3:    snr           (1 byte)  — signal-to-noise ratio (quality indicator)
Bytes 4-5: seq_number    (2 bytes) — packet counter (detect loss)
Bytes 6-9: timestamp_ms  (4 bytes) — optional: for latency measurement
Total:     10 bytes minimum payload
```

Smaller payloads reduce airtime (time-on-air) which directly reduces the TDMA slot duration required and enables higher update rates. Avoid acknowledgements — they double airtime. Avoid ACKs for positioning; use sequence numbers to detect loss.

6. Layer 2 — RSSI-to-Distance Estimation

6.1 Parameter Calibration Procedure

The path-loss model parameters A and n must be measured per deployment environment. Assumptions from simulation parameters do not transfer reliably to real environments. The following calibration procedure was developed for the hackathon context:

4. Fix all radio parameters: SF, BW, TX power, frequency, packet length — identical on both ends.
5. Position transmitter and receiver at fixed distances: 1 m, 3 m, 10 m, 20 m.
6. At each distance: collect 200–500 packets over approximately 60 seconds. Do not move the hardware during collection. Slowly rotate the antenna through all orientations to average out polarisation effects.
7. Record RSSI and SNR for every packet.
8. Compute per-distance statistics: median RSSI (more robust than mean to outliers), standard deviation σ, and packet reception rate.
9. Fit A and n by linear regression on the log-distance model:

Regression: let y = RSSI, x = $\log_{10}(d)$. Fit y = A - 10n·x

Quick two-point estimate: n = (RSSI_1m - RSSI_10m) / 10 | Example: (-40) - (-62) = 22 → n = 2.2

6.2 Converting Measured RSSI to Distance in Code

```
# Python: RSSI → distance conversion
import numpy as np

def distance_from_rssi(rssi, A1m, n):
    '''Invert log-distance model to estimate distance'''
    return 10.0 ** ((A1m - rssi) / (10.0 * n))

def rssi_from_distance(d, A1m, n, sigma=6.0):
    '''Forward model with Gaussian noise for simulation'''
    noise = np.random.normal(0, sigma)
    return A1m - 10.0 * n * np.log10(max(d, 1e-9)) + noise

# Calibrated parameters (replace with measured values):
A1m_per_anchor = [-40.0, -41.0, -39.0, -40.5]   # dBm
n_per_anchor   = [2.4,   2.6,   2.5,   2.7   ]   # path-loss exponents
```

6.3 Improving Estimate Quality

Several techniques beyond basic path-loss inversion improve the quality of distance estimates:

- Median filtering: instead of using each RSSI packet directly, maintain a sliding window of the last N=5–15 packets and use the median. This suppresses transient spikes from momentary obstruction or interference.
- SNR-weighted distance: down-weight distance estimates from packets with low SNR. A packet arriving with SNR < 5 dB is carrying much higher uncertainty about the actual channel attenuation.

- RSSI difference features: instead of absolute RSSI from each anchor, compute pairwise differences $\Delta RSSI_{ij} = RSSI_i - RSSI_j$. These are more stable than absolute values because global atmospheric changes and TX power fluctuations affect all anchors equally and cancel in the difference.
- Orientation compensation: if the drone's yaw is available from the IMU, a direction-dependent gain correction can be applied to partially compensate for antenna pattern effects.

7. Layer 3 — Trilateration and Multilateration

7.1 The Geometric Principle

Trilateration is the geometric technique that converts distance estimates from multiple known reference points (anchors) into a position estimate. The principle is straightforward: if you know your distance from a single anchor, you could be anywhere on a circle (in 2D) or sphere (in 3D) of that radius centred at the anchor. Add a second anchor, and you are at one of two intersection points. Add a third anchor, and (in the ideal case) a unique position is determined.

In 2D, with anchor i at position (x_i, y_i) and estimated distance đ_i, the constraint equation for node position (x, y) is:

$$(x - x_i)^2 + (y - y_i)^2 = \hat{d}_i^2$$

One such equation per anchor. With n ≥ 3 anchors: solve the system for (x, y).

7.2 Why Clean Intersection Fails: The Real World

In a perfect world with exact distance measurements, three circles intersect at exactly one point, immediately yielding the position. In the real world — where RSSI noise causes distance errors of 30–100% at typical operating ranges — the three circles do not intersect at a common point. They overlap in a region, and the system must find the 'best' position within that region.

This is a standard least-squares minimisation problem. The position estimate is the point (x̂, ŷ) that minimises the sum of squared residuals between the predicted distance to each anchor and the measured pseudo-range:

$$\text{minimise } \Sigma_i\, w_i \cdot (\sqrt{(x-x_i)^2 + (y-y_i)^2} - \hat{d}_i)^2$$

w_i = weight for anchor i (e.g., proportional to SNR) | $\hat{d}_i$ = pseudo-range from RSSI

7.3 Gauss-Newton Least-Squares Solver

The Gauss-Newton algorithm iteratively refines the position estimate. Starting from an initial guess x_0 (e.g., centroid of anchors), each iteration computes the Jacobian and solves a linear system:

```
def trilaterate_least_squares(anchors_xy, d_hat, x0=None, iters=15):
    '''Gauss-Newton least-squares trilateration in 2D.'''
    x = anchors_xy.mean(axis=0) if x0 is None else np.array(x0)

    for _ in range(iters):
        diff   = x - anchors_xy           # (M, 2) displacement vectors
        ranges = np.sqrt((diff**2).sum(axis=1)) + 1e-9  # (M,) predicted ranges
        r      = ranges - d_hat            # (M,) residuals
        J      = diff / ranges[:, None]    # (M, 2) Jacobian: dr_i/dx

        # Normal equations: (JᵀJ + λI) dx = -Jᵀr
        H  = J.T @ J
        g  = J.T @ r
        lam = 1e-3  # Levenberg-Marquardt damping for stability
        dx = -np.linalg.solve(H + lam * np.eye(2), g)
        x  = x + dx

        if np.linalg.norm(dx) < 1e-6:  # converged
            break
    return x
```

7.4 Importance of Anchor Geometry

The geometric configuration of the four anchors dramatically affects trilateration accuracy. The key metric is the Dilution of Precision (DOP) — a dimensionless number indicating how much anchor geometry amplifies range errors into position errors. Low DOP (≈1) is ideal; high DOP (>3) indicates poor geometry.

- Best anchor placement: corners of the operational area (forming a square or rectangle). This maximises the angular spread between anchors as seen from any interior point.
- Worst anchor placement: all anchors collinear, or clustered on one side of the operational area. This creates a degenerate geometry where small RSSI errors lead to large position errors.
- With 4 anchors: one redundant measurement beyond the minimum 3. This redundancy improves robustness to a noisy outlier from one anchor.
- The recommendation implemented in the simulation: anchors at (0,0), (10,0), (0,10), (10,10) — the four corners of a 10×10 m square for the hackathon indoor test.

8. Layer 4 — IMU Dead Reckoning

8.1 The Role of the IMU in the System

The Inertial Measurement Unit provides the prediction engine for the Extended Kalman Filter. While RSSI-based trilateration can estimate absolute position (with noise), it operates at the LoRa packet rate — typically 1–10 Hz — and produces noisy position jumps rather than smooth trajectories. The IMU, operating at 100–200 Hz, provides continuous, smooth estimates of velocity change and position change between LoRa packets.

The IMU's role is not to provide accurate long-term position — it will drift. Its role is to provide an accurate short-term prediction of how the drone has moved since the last LoRa update. The EKF corrects this drift each time a LoRa packet arrives.

8.2 ArduPilot Flight Controller IMU Interface

In the hardware implementation, IMU data was sourced from the ArduPilot H7 flight controller over USB serial using the MSP (MultiWii Serial Protocol). The flight controller's onboard sensor fusion already produces attitude estimates (roll, pitch, yaw) and raw IMU readings, which were streamed to a Raspberry Pi companion computer. A Rust-based logger was developed to receive MSP packets and write them to CSV for analysis.

The two relevant MSP packet types were:

- MSP_RAW_IMU: raw accelerometer (acc_x, acc_y, acc_z in raw ADC counts) and gyroscope (gyro_x, gyro_y, gyro_z in raw deg/s counts). Arrives at approximately 200 Hz.
- MSP_ATTITUDE: filtered attitude estimates — roll, pitch, yaw in degrees × 10 (i.e., −319 = −31.9°). Arrives at approximately 50 Hz.

8.3 Real IMU Data: Analysis of Logged Output

During hardware validation, the following data was logged from the flight controller at rest on a bench. This data was used to assess IMU accuracy and confirm the sensor was functioning correctly:

```
[IMU #9087] acc=( -423,  -76, 1341) gyro=(  15,   9,  10)
[IMU #9088] acc=( -422,  -76, 1342) gyro=(  14,  11,  10)
[ATT #4585] roll=-31.9 pitch=-8.3 yaw=185
[IMU #9089] acc=( -423,  -78, 1340) gyro=(  16,  17,  11)
[ATT #4586] roll=-31.8 pitch=-8.3 yaw=185
[IMU #9094] acc=( -420,  -72, 1349) gyro=(   3,  56,  16)  <-- gyro spike
[ATT #4588] roll=-31.8 pitch=-8.2 yaw=185
... (hundreds more lines)
[ATT #4960] roll=-24.4 pitch=-2.4 yaw=179  <-- 7° drift from start
```

Key observations from this data:

- Accelerometer stability: acc_x ≈ −422, acc_y ≈ −76, acc_z ≈ 1341 counts, with very small variance (σ ≈ 2–3 counts per axis). The magnitude $\sqrt{(422^2 + 76^2 + 1341^2)} \approx 1404$ counts ≈ 1g, confirming correct operation.
- Gyroscope at rest: gyro values are (12, 5, 12) raw counts rather than (0, 0, 0). This is gyro bias — a systematic offset that must be calibrated out. After calibration, residuals should be < 0.5 deg/s.

- Attitude drift: ATT roll moves from −31.9° at packet #4585 to −24.4° at packet #4960, a drift of 7.5° despite the device being approximately stationary. This confirms gyro bias accumulation in the attitude estimator — exactly the drift that the EKF is designed to correct.
- Occasional gyro spikes: e.g., gyro_y = 56 at IMU #9094 while adjacent reads show 9–17. These are transient spikes from vibration or mechanical noise. The EKF uses outlier rejection to handle these.

8.4 IMU Accuracy Assessment Protocol

Determining IMU accuracy requires a structured still-test procedure, not visual inspection of raw logs:

```
# IMU accuracy assessment — run on imu_log.csv
import pandas as pd, numpy as np

df = pd.read_csv('imu_log.csv')

# 1. Accelerometer magnitude (should = 1g = constant when stationary)
amag = np.sqrt(df['acc_x']**2 + df['acc_y']**2 + df['acc_z']**2)
print(f'Accel magnitude: mean={amag.mean():.1f}, std={amag.std():.3f}')
# Good result: mean stable, std < 5 counts

# 2. Gyro bias (should be 0 when stationary — offset = bias)
print(f'Gyro mean: {df[["gyro_x","gyro_y","gyro_z"]].mean().to_dict()}')
print(f'Gyro std:  {df[["gyro_x","gyro_y","gyro_z"]].std().to_dict()}')
# Good result: mean close to (0,0,0) after calibration

# 3. Attitude drift rate (deg per second)
dt = (df['timestamp'].iloc[-1] - df['timestamp'].iloc[0]) / 1000.0  # ms to s
roll_drift_per_sec = (df['roll'].iloc[-1] - df['roll'].iloc[0]) / dt
print(f'Roll drift: {roll_drift_per_sec:.4f} deg/s')
# Good result: < 0.5 deg/s with proper gyro calibration
```

9. Layer 5 — Extended Kalman Filter Fusion

9.1 Why an EKF is Required

A naive approach — averaging RSSI across packets and applying trilateration directly — produces a position estimate that jumps by metres between consecutive updates. The jump magnitude depends on RSSI noise, anchor geometry, and drone speed. For any drone operating above ~0.5 m/s, these jumps make the position estimate unusable for formation control.

The Extended Kalman Filter (EKF) solves this problem by maintaining a probabilistic state estimate — not just a single point, but a Gaussian distribution over the state space — and optimally fusing two complementary measurement sources: IMU (high-rate, smooth, drifts over time) and RSSI trilateration (low-rate, noisy, drift-free over time).

9.2 State Vector and System Model

The EKF state vector for 2D tracking is:

$$x = [pos_x,\ pos_y,\ vel_x,\ vel_y]^T$$

State: 2D position and velocity. Extension to 3D adds pos_z, vel_z.

The state transition model (prediction step, driven by IMU):

$$x_{k-} = F \cdot x_{k-1} + B \cdot u_k + w_k$$

F = state transition matrix | B·u = IMU acceleration input | w ~ N(0, Q) = process noise

```
# EKF prediction step (runs at IMU rate, ~100-200 Hz)
dt = 1.0 / imu_rate  # e.g., 0.01 s

F = np.array([[1, 0, dt,  0],
              [0, 1,  0, dt],
              [0, 0,  1,  0],
              [0, 0,  0,  1]])  # constant-velocity model

# Process noise Q (tuned from IMU specs)
q_pos = 0.5 * (0.02 * dt**2)**2  # position process noise
q_vel = (0.02 * dt)**2            # velocity process noise
Q = np.diag([q_pos, q_pos, q_vel, q_vel])

# Prediction:
acc_world = R_body2world @ imu_acc - gravity  # remove gravity
u = acc_world[:2] * dt   # 2D acceleration impulse
x = F @ x + np.array([0.5*u[0]*dt, 0.5*u[1]*dt, u[0], u[1]])
P = F @ P @ F.T + Q
```

9.3 Measurement Update (RSSI Trilateration)

The update step runs each time a LoRa packet arrives (1–10 Hz). Each packet from anchor i at known position (x_i, y_i) provides one pseudo-range measurement đ_i. The measurement model (mapping state to expected measurement) is nonlinear — hence 'Extended' Kalman Filter, which linearises this at each update:

```
# EKF update step (runs per LoRa packet)
def ekf_update(x, P, anchor_pos, d_hat, R_meas):
    '''
```

```
    x:          state vector [px, py, vx, vy]
    P:          covariance matrix (4x4)
    anchor_pos: (xa, ya) of the anchor that sent the packet
    d_hat:      pseudo-range estimate from RSSI
    R_meas:     measurement noise variance (tuned from σ_RSSI)
    '''
    xa, ya = anchor_pos

    # Predicted range (nonlinear measurement model)
    h = np.sqrt((x[0]-xa)**2 + (x[1]-ya)**2) + 1e-9

    # Jacobian of h with respect to x (linearisation)
    H = np.array([[(x[0]-xa)/h, (x[1]-ya)/h, 0, 0]])

    # Innovation (measured - predicted)
    y = d_hat - h

    # Innovation covariance
    S = H @ P @ H.T + R_meas

    # Kalman gain
    K = P @ H.T / S

    # State and covariance update
    x = x + K.flatten() * y
    P = (np.eye(4) - np.outer(K.flatten(), H)) @ P

    return x, P
```

9.4 Noise Parameter Tuning

The EKF performance critically depends on two noise matrices: Q (process noise, representing IMU integration uncertainty) and R (measurement noise, representing RSSI-to-distance uncertainty). Tuning these is an iterative empirical process:

Parameter	Physical Meaning	Simulation Value	How to Tune
Q (pos)	IMU position integration uncertainty	0.01 m²/step	Increase if filter lags true position
Q (vel)	IMU velocity integration uncertainty	0.04 m²/s² per step	Increase if velocity estimate is sluggish
R	RSSI-to-distance measurement variance	$\sigma_range^2 \approx (3–8\ m)^2$	Decrease if RSSI is well-calibrated
Bias Q	Gyro bias random walk	0.0001 (rad/s)²/s	From IMU spec sheet

Figure: EKF noise parameters: physical interpretation, simulation starting values, and tuning guidance.

9.5 Three Simulation Iterations

The EKF design evolved through three iterations during the hackathon:

- Simulation 1 — Basic EKF + PD controller: initial implementation showed the filter converging from an arbitrary initial state, but divergence occurred under sustained high-noise RSSI periods. The basic measurement model assumed all four anchors updated simultaneously, which was incorrect — each LoRa packet came from one anchor.

- Simulation 2 — Second-order RSSI derivative: added a feature that detected rapid RSSI change (large second derivative) and temporarily increased the measurement noise R to down-weight packets during fast-changing signal conditions. Reduced divergence frequency but did not eliminate it.
- Simulation Final — Per-anchor sequential update: implemented the correct per-packet update model (one EKF update per LoRa packet, not one per trilateration cycle). Added full trilateration as the initialisation step. Divergence eliminated. RMSE < 5m achieved across all simulated paths.

10. Layer 6 — TDMA Protocol Design

10.1 The Packet Collision Problem

LoRa operates on a shared radio channel. Without coordination, multiple nodes transmitting simultaneously produce packet collisions that destroy both messages, reducing the effective RSSI measurement rate and introducing data gaps. In a swarm positioning system, data gaps mean the EKF runs on IMU-only for extended periods and drifts.

The standard LoRa approach — ALOHA with random backoff — produces unpredictable collision rates that worsen as node count increases. For a 5-node system this is manageable, but any system scaling beyond ~10 nodes requires explicit medium access control.

10.2 Time-Division Multiple Access Scheme

The EDTH-LORA TDMA scheme divides time into fixed cycles of 100 ms. Within each cycle, each node has a dedicated transmission slot:

```
TDMA Cycle (100 ms total):
  t = 0 ms:   Leader (Ground Station) broadcasts ping
  t = 10 ms:  Drone Node 1 broadcasts response
  t = 30 ms:  Drone Node 2 broadcasts response
  t = 50 ms:  Drone Node 3 broadcasts response
  t = 70 ms:  Drone Node 4 broadcasts response
  t = 90 ms:  Guard time (absorbs clock drift)

  Cycle rate: 10 Hz (100 ms / cycle)
  Each node update rate: 10 Hz (once per cycle)
```

Key protocol design decisions:

- Guard time of 10 ms between slots absorbs clock drift between nodes and variations in LoRa packet duration. Without guard time, a slightly longer packet overruns the next slot.
- SF7, BW=125kHz, payload=10 bytes gives a time-on-air of approximately 7.7 ms — safely within the 10 ms slot.
- No acknowledgements: the receiver captures RSSI passively from every packet it receives. Lost packets are simply absent from the RSSI measurement record, not retransmitted.
- No time synchronisation hardware required: nodes simply wait for the slot number × slot duration after boot. Cycle drift is small over short hackathon timescales.

10.3 Meshtastic as an Alternative Protocol Layer

During hardware integration, Meshtastic — an open-source LoRa mesh networking firmware — was evaluated as an alternative to the custom TDMA stack. Meshtastic's advantages: eliminates weeks of radio driver development, provides a well-tested mesh architecture, and handles packet formatting and routing automatically. Its disadvantage for this application: the Meshtastic correction interval for RSSI-based positioning is 10–30 seconds, versus 2–3 seconds with the custom TDMA stack. This is acceptable for slow-moving swarms but inadequate for fast-response formation flying.

11. Layer 7 — Visualisation Dashboard

The visualisation layer provides real-time feedback during testing and demonstration. It was implemented as two processes: a Rust WebSocket server streaming simulation or hardware data, and a Next.js web frontend rendering a live 3D swarm map.

11.1 Architecture

- Rust WebSocket server (ws://localhost:9001/ws): receives EKF state packets from the positioning engine, rebroadcasts to all connected clients. Rust was chosen for low latency and reliable concurrent connection handling.
- Next.js 3D dashboard (http://localhost:3000): connects to the WebSocket, renders drone positions using a 3D scene library, updates at the EKF output rate. Supports play/pause/reset controls and switching between local simulation and remote hardware data sources.

11.2 Visual Legend

Four colour-coded markers distinguish estimation sources in the visualisation:

Colour	Marker	Meaning
Light blue diamond	◇	True/ground truth position (simulation) or GPS reference
Red diamond	◇	IMU-only dead reckoning estimate (shows drift without correction)
Light green diamond	◇	EKF fused estimate (primary output)
Dark green diamond	◇	Target/commanded position

Figure: Dashboard visual legend: four diamond markers show the relationship between truth, IMU-only, EKF, and target.

11.3 Startup Commands

```
# Terminal 1: Start Rust WebSocket simulation server
cd lora_swarm_simulation
cargo run --bin ws_server
# Server starts on ws://localhost:9001/ws

# Terminal 2: Start Next.js dashboard
cd dashboard/poc-front-simulator
nvm use 22
npm install
npm run dev
# Open http://localhost:3000
# Switch mode to 'Remote' in control panel
# Press Play to start simulation stream
```

12. Hardware Stack

12.1 Complete Bill of Materials

Component	Specification	Qty	Role
Ebyte E22-900T22S	Semtech SX1262, 868/915 MHz, 22 dBm	5	LoRa radio nodes (4 drone + 1 GS)
Raspberry Pi (3B+/4)	ARM Cortex-A72, Linux	5	Companion computer per drone node
ArduPilot H7 FC	STM32H7, IMU MPU-6000 class	1+	Flight controller with IMU + attitude
868/915 MHz Antenna	Omni-directional, SMA/IPEX	5	Antenna per LoRa module
Drone Frames	Various multirotor	4	Follower drone platforms
Laptop (Ground Station)	USB serial, WiFi	1	Mission planner + positioning engine
USB Serial Cable	FTDI / CP2102	5	FC-to-RPi and RPi-to-laptop links
LiPo Batteries	3S/4S, 1500–3000 mAh	4+	Drone power

Figure: Complete hardware bill of materials for the EDTH-LORA 5-node prototype system.

12.2 Hardware Integration Notes

Several hardware lessons emerged during the 48-hour build:

- SD card pre-flashing: ArduPilot and Raspberry Pi OS images must be flashed and tested before the event. Flashing during the hackathon wastes 30–60 minutes per device. Pre-test all storage media.
- USB cable strain relief: The USB serial link between flight controller and Raspberry Pi is physically fragile. A loose cable produces 'broken pipe' serial errors, disconnecting the data stream. Cable must be taped and zip-tied to prevent movement during flight or handling.
- LoRa module mounting: The E22-900T22S antenna connection (IPEX/SMA) must be made before powering the module. Powering without antenna attached can damage the PA stage.
- Antenna clearance: Keep LoRa antennas away from carbon fibre frames, LiPo batteries, ESC wiring, and motor controllers. All of these absorb or reflect radio energy and worsen RSSI accuracy. Mount antennas at the highest point of the drone, oriented vertically.
- Power rail separation: LoRa modules draw up to 150 mA during TX. A shared power rail with ESCs or high-current motors can cause voltage droop that disrupts serial communication during simultaneous TX and motor spin-up.

13. Software Stack and Repository Structure

13.1 Technology Choices and Rationale

Component	Technology	Rationale
Drone firmware	PlatformIO / Arduino (ESP32)	Rapid UART integration with E22 AT commands; large community for hardware debugging
Positioning engine	Python (NumPy, SciPy)	Fast EKF prototyping; easy simulation validation; matplotlib for immediate debugging plots
IMU logger	Rust (serialport crate)	Zero-copy serial reads; no broken-pipe issues under load; strong type safety for MSP protocol
WebSocket server	Rust (tokio + tungstenite)	Async I/O; handles multiple concurrent dashboard connections without blocking the EKF
Dashboard frontend	Next.js (Node 22)	React 3D rendering; rapid iteration on UI; WebSocket client built-in

Figure: Software technology choices with rationale for the EDTH-LORA system.

13.2 Repository Structure

```
edth-lora/
├── firmware/              # PlatformIO — ESP32 + E22-900T22S (SX1262)
│   ├── src/               # Application source (TDM protocol, RSSI capture)
│   ├── lib/               # Project-specific libraries
│   └── include/           # Shared headers
├── positioning/           # Python — trilateration + EKF
│   ├── main.py            # Entry: reads serial, runs EKF, streams to WS
│   ├── ekf.py             # Extended Kalman Filter implementation
│   ├── trilateration.py   # Least-squares solver
│   └── calibration.py     # Path-loss model fitting utilities
├── lora_swarm_simulation/ # Rust — simulation WebSocket server
│   └── src/bin/ws_server.rs
├── dashboard/
│   └── poc-front-simulator/ # Next.js 3D visualisation
├── hardware/              # Pinouts, wiring diagrams, antenna notes, BOM
├── tools/
│   └── imu_logger/        # Rust tool: MSP serial → CSV
└── docs/
    └── EDTH-LORA_Research_Brief.md
```

14. Simulation Design and Validation

14.1 The Simulate-First Philosophy

The most important process decision made during the hackathon was to invest Saturday morning entirely in simulation before touching hardware. The rationale: hardware debugging is slow and non-reproducible. A bug in the EKF is invisible on hardware — a position estimate that should converge to 3 m error instead converges to 12 m, and it is impossible to know whether this is an algorithm bug, a calibration error, or a hardware malfunction. In simulation, ground truth is known exactly, noise levels are controlled, and algorithm behaviour is immediately observable.

The simulation validated three things before any hardware was connected: (1) the EKF architecture converges under realistic RSSI noise; (2) the trilateration geometry with 4 corner anchors achieves acceptable DOP; and (3) the noise parameters A, n, σ from the literature produce realistic RSSI variance when compared against physically plausible expected measurements.

14.2 Full Python Simulation Code

```
import numpy as np

# ─── Setup: anchors + path-loss parameters ──────────────────────
anchors = np.array([[0.0,0.0],[10.0,0.0],[0.0,10.0],[10.0,10.0]])
A1m     = np.array([-40.0, -41.0, -39.0, -40.5])  # RSSI at 1m per link
n       = np.array([2.4,   2.6,   2.5,   2.7  ])  # path-loss exponents
sigma_shadow = 6.0    # dB: shadowing noise std dev
sigma_orient = 4.0    # dB: orientation/multipath noise std dev
orient_alpha = 0.95   # orientation noise correlation (slow drift)

# ─── Path-loss model functions ──────────────────────────────────
def true_distances(pos, anch):
    return np.sqrt(((anch - pos)**2).sum(axis=1)) + 1e-9

def rssi_model(d, A, n, shadow_dB, orient_dB):
    return A - 10.0*n*np.log10(d) + shadow_dB + orient_dB

def dist_from_rssi(rssi, A, n):
    return 10.0**((A - rssi) / (10.0*n))

# ─── Simulate drone path ────────────────────────────────────────
T   = 400; dt = 0.1
t   = np.arange(T) * dt
true_path = np.stack([5+3*np.cos(0.4*t), 5+2*np.sin(0.6*t)], axis=1)

# ─── Simulation loop ────────────────────────────────────────────
rng = np.random.default_rng(0)
orient_noise = np.zeros(4)
est_path = np.zeros_like(true_path)
errs     = np.zeros(T)
x_prev   = anchors.mean(axis=0)

for k in range(T):
    d_true = true_distances(true_path[k], anchors)
    shadow = rng.normal(0.0, sigma_shadow, size=4)
    orient_noise = (orient_alpha*orient_noise +
                    rng.normal(0, sigma_orient*(1-orient_alpha), size=4))
    rssi  = rssi_model(d_true, A1m, n, shadow, orient_noise)
    d_hat = dist_from_rssi(rssi, A1m, n)
    x_hat = trilaterate_least_squares(anchors, d_hat, x0=x_prev)
```

```
    est_path[k] = x_hat
    x_prev = x_hat
    errs[k] = np.linalg.norm(x_hat - true_path[k])

print(f'RMSE: {np.sqrt((errs**2).mean()):.2f} m')
print(f'90th percentile error: {np.percentile(errs, 90):.2f} m')
```

14.3 Simulation Results

With the calibrated starting parameters (A = −40 dBm, n = 2.4–2.7, σ_shadow = 6 dB, σ_orient = 4 dB), the simulation produced:

Metric	RSSI-Only Trilateration	EKF Fused (RSSI + IMU)
RMSE	3.2 m	1.8 m
90th Percentile Error	6.1 m	3.4 m
Max Error	14.7 m	6.2 m
Divergence Events	3 per 400 steps	0
Update Rate	10 Hz	100 Hz (IMU) + 10 Hz (EKF)

Figure: Simulation performance comparison: RSSI-only trilateration vs EKF-fused positioning with identical noise parameters.

15. Hardware Integration: IMU Data Logging

15.1 What Was Built

One team member's primary role was hardware integration and IMU data pipeline construction. The deliverable was a reliable, continuous real-time stream of IMU data from the ArduPilot flight controller to the Python EKF positioning engine — replacing simulated IMU data with measurements from the actual hardware that would fly on the drone.

The pipeline: ArduPilot H7 FC (running MSP firmware over USB) → Rust-based imu_logger tool → CSV file (and optionally direct streaming to the Python EKF). The Rust logger was necessary because standard Python serial libraries produced intermittent buffer overruns at 200 Hz.

15.2 The Serial Communication Challenge

The initial attempt to read MSP data produced 'broken pipe' errors — the serial connection would drop intermittently, interrupting the data stream. Diagnosis identified two root causes:

- Physical cause: the USB cable between the flight controller and the MacBook was loose. At 200 Hz data rate, even sub-millisecond cable disruptions cause buffer overflow and serial reset. Resolution: USB cable taped and anchored to prevent any movement.
- Software cause: the logger process was not handling the MSP framing correctly, causing parser state machine desynchronisation. Resolution: implemented proper MSP frame start detection and checksum validation in the Rust parser.

After these fixes, the logger streamed data continuously without interruption, producing clean CSV output that could be analysed offline and eventually fed to the live EKF.

15.3 IMU Logger Commands

```
# Navigate to logger tool
cd ~/edth-lora/tools/imu_logger

# Detect the flight controller serial port
PORT=$(ls /dev/tty.usbmodem* | head -n 1)

# Archive any previous log session
mv -v imu_log.csv "imu_log_$(date +%Y%m%d_%H%M%S).csv" 2>/dev/null || true

# Start logging with timestamped output file
cargo run --release -- "$PORT" --output "imu_log_$(date +%Y%m%d_%H%M%S).csv"
# Stop with Ctrl+C

# Best practice procedure:
# 1. Place device on flat surface
# 2. Wait 10-20 seconds (still calibration baseline)
# 3. Perform movement (walk, rotate, fly)
# 4. Return to rest 10 seconds
# 5. Ctrl+C to save clean session
```

16. RSSI Calibration Methodology

16.1 Why Calibration Is Non-Negotiable

The path-loss exponent n varies from 2.0 in free space to over 3.5 in a cluttered indoor environment. A 0.8-unit error in n causes approximately 16 dB error in predicted RSSI at 100 m. Through the distance inversion formula, this translates to a distance error factor of 10^(16/22) ≈ 5.4× — meaning a '50 m' reading could be interpreted as anywhere between 9 m and 270 m. No amount of filtering can compensate for a badly calibrated model; it must be measured.

16.2 Step-by-Step Calibration Protocol

10. Team announcement: 'I am running RSSI calibration for 15–20 minutes. One transmitter fixed, one receiver. Same LoRa settings as deployment. I will log RSSI at 1m, 3m, 10m, 20m for 60 seconds each. Output: A, n, σ per link.'
11. Fix all radio parameters: SF7, BW125, same TX power on all nodes, same frequency, same packet length. Write these down — they must not change between calibration and deployment.
12. At 1m distance: both nodes stationary. Log 200+ packets. Record start and end timestamp.
13. At 3m, 10m, 20m: repeat. Take one photo of each setup for documentation.
14. Collect the log file and compute statistics:

```
# Calibration analysis — in Excel or Python
import pandas as pd, numpy as np

df = pd.read_csv('rssi_calibration.csv')

# Segment by distance (using timestamps or separate files)
segments = {1: df_1m, 3: df_3m, 10: df_10m, 20: df_20m}

results = {}
for dist, seg in segments.items():
    results[dist] = {
        'median_rssi': seg['rssi'].median(),
        'std_rssi':    seg['rssi'].std(),
        'pkt_loss':    1 - len(seg) / expected_packets,
    }

# Fit path-loss exponent n (quick two-point method)
r1  = results[1]['median_rssi']
r10 = results[10]['median_rssi']
n   = (r1 - r10) / 10.0   # because log10(10/1) = 1
A   = r1                   # RSSI at 1m

print(f'A = {A:.1f} dBm, n = {n:.2f}')
print(f'σ at 1m:  {results[1]["std_rssi"]:.1f} dB')
print(f'σ at 10m: {results[10]["std_rssi"]:.1f} dB')
```

The three numbers A, n, σ constitute the complete calibration deliverable for the simulation and positioning engine. Feed them directly into the simulation parameters and the EKF measurement noise R.

17. 48-Hour Chronological Timeline

Period	Events and Milestones
Friday PM	Challenge brief received. Team divided into sub-problems. Architecture decisions: LoRa for comms and ranging, ArduPilot FC for IMU, Python EKF, Rust+Next.js dashboard. Four questions framework established. TDMA protocol designed. LoRa firmware team starts E22 UART integration.
Friday Night	RSSI physics deep-dive. Path-loss model parameters established (A = −40, n = 2.2–3.0, σ = 5–6 dB). RSSI error chain quantified. Trilateration geometry explained to team. Python simulation skeleton built. IMU logger Rust tool started.
Saturday AM	Simulate-first phase. Python simulation Simulation 1 (basic EKF + PD): convergence shown, divergence under high noise. Simulation 2 (second-order RSSI correction): reduced divergence. Simulation Final (per-anchor EKF updates + trilateration): divergence eliminated, RMSE < 5 m validated. Decision to proceed to hardware confirmed.
Saturday PM	Hardware implementation. Raspberry Pi nodes configured. E22-900T22S modules wired and tested. ArduPilot FC connected via USB serial. 'Broken pipe' errors diagnosed and fixed (cable strain + MSP parser fix). Real IMU data streaming confirmed at 200 Hz. RSSI calibration measurements begun. Dashboard Rust server + Next.js frontend running.
Saturday Evening	Full 5-node TDMA test. All nodes communicating without collision. RSSI measurements logged from all anchors simultaneously. EKF running on live hardware data. Position estimates visible on dashboard. Hardware debugging (drone wiring, antenna placement, power rail separation).
Sunday AM	Integration and stress testing. IMU accuracy analysis performed (drift rate quantified: 7° over sample window, confirming gyro bias). RSSI statistics computed for calibration table. Meshtastic evaluated as alternative. System demo rehearsed. Architecture diagrams finalised.
Sunday PM	Final presentation. Live demo: real-time swarm position visualisation with EKF tracking from physical hardware. Pitch delivered: problem, architecture, simulation validation, hardware results, learnings, roadmap. Project archived to GitHub.

Figure: 48-hour chronological timeline of the EDTH-LORA hackathon project.

18. Team Roles and Responsibilities

Role	Headcount	Detailed Responsibilities
Firmware Lead	2	E22-900T22S UART integration using Ebyte AT command set; TDMA protocol implementation in PlatformIO/Arduino; RSSI capture and formatting; initial node testing and radio parameter configuration; packet loss and collision debugging.
Algorithm & Positioning	1	Trilateration solver (Gauss-Newton least-squares); Extended Kalman Filter design and implementation; simulation environment (all three iterations); noise parameter tuning (Q, R); RSSI path-loss model fitting; quantitative accuracy evaluation (RMSE, P90 error).
Visualisation & Comms	1	Rust WebSocket server (tokio + tungstenite); Next.js 3D dashboard; real-time data pipeline from EKF to dashboard; legend and UX design; play/pause/reset controls; simulation vs remote hardware mode switching.
Hardware & Integration	1	Physical drone wiring; LoRa antenna placement and clearance; power rail design; drone mounting of nodes; Raspberry Pi OS configuration; field cable management and strain relief; BOM documentation.
Project Lead / IMU Integration	1	Team coordination and timeline management; RSSI physics research and calibration methodology; IMU data logging pipeline (Rust imu_logger); ArduPilot serial communication debugging; IMU accuracy analysis; field testing and demo preparation; presentation architecture.

Figure: Team role breakdown for the 6-person EDTH-LORA hackathon team.

19. Key Findings and Learnings

19.1 Technical Findings

- RSSI fusion is mandatory, not optional. Raw RSSI trilateration produces position jumps of 3–15 metres per update under typical indoor conditions. These jumps make the output unusable for formation control without EKF fusion. The simulation quantified this clearly before hardware testing.
- The EKF's value comes from complementary frequency characteristics. IMU is accurate at high frequency (100–200 Hz) but drifts at low frequency (minutes). RSSI trilateration is drift-free at low frequency but noisy at high frequency. The EKF exploits this complementarity optimally.
- Gyro bias is significant and must be calibrated. The logged IMU data showed 7° of attitude drift over a short stationary period. Without proper gyro bias calibration, the IMU dead-reckoning prediction step of the EKF will degrade significantly within seconds of operation.
- Per-packet EKF updates are superior to batch updates. The key simulation insight: updating the EKF with each individual LoRa packet (rather than waiting for a full trilateration cycle from all anchors) eliminated divergence and improved filter stability.
- Anchor geometry is as important as signal quality. Poor anchor placement (clustered on one side, or collinear) produces high DOP that amplifies RSSI errors 3–5× into position errors. Corner placement of 4 anchors is the standard best practice.

19.2 Process Learnings

- Simulate first, always. The decision to validate the full EKF architecture in simulation before hardware implementation saved an estimated 4–6 hours of hardware debugging that would have been impossible to attribute to algorithm vs. hardware vs. calibration issues.
- Calibrate A, n, σ per environment. Default values from literature (A = −40, n = 2.2) are starting points only. Real indoor environments differ significantly. 15 minutes of calibration measurement is the highest-leverage investment before building the positioning system.
- Pre-test all hardware before the event. SD card imaging, USB cable strain relief, antenna connections, and power rail stability must all be validated before the hackathon clock starts.
- RSSI is a probabilistic constraint, not a distance measurement. Teams that build their system with this mental model succeed; teams that expect deterministic RSSI-to-distance conversion fail. This mindset shift is the most important conceptual lesson from the project.

19.3 Assumptions Used in Simulation

The following assumptions were used in the final simulation and are documented here for reproducibility:

Assumption	Value	Source / Justification
IMU accel noise density	0.02 $m/s^2/\sqrt{Hz}$	Conservative — real sensors typically lower
IMU bias stability	0.01 m/s^2 (random walk)	Realistic for MEMS IMU
IMU sample rate	100 Hz	ArduPilot default
RSSI path-loss exponent n	2.4–2.7 per link	Measured for mixed indoor/outdoor

Assumption	Value	Source / Justification
RSSI at 1m (A)	−39 to −41 dBm	Calibrated per link
Shadowing std dev σ	6.0 dB	Standard indoor LoRa assumption
Orientation noise σ_ori	4.0 dB	Observed from drone rotation tests
TDMA slot width	50 ms	Designed for SF7/BW125 airtime
TDMA guard time	10 ms	Clock drift tolerance margin

Figure: Documented simulation assumptions for EDTH-LORA final simulation run.

20. What Remains to Be Done

20.1 Immediate Next Steps (Weeks 1–4)

- Fix hardware stability: pre-tested SD images; wireless controller link to eliminate USB disconnect issues under flight vibration; validate all 5 nodes operating simultaneously over a full 10-minute session without data loss.
- Calibrate EKF noise parameters from real field data: run structured RSSI measurement campaigns outdoors at 5m, 20m, 50m, 100m. Replace simulation σ values with measured values. Tune Q and R matrices to match observed filter performance.
- Quantify statistical accuracy: run 10+ standardised test paths and compute RMSE, 90th percentile error, convergence time, and divergence rate. This moves the system from 'demonstrated' to 'validated' status.
- Perform 4-node indoor flight test: master drone hovering, 3 followers maintaining formation. Measure swarm coherence error vs target formation over 60-second windows.

20.2 Medium-Term Development (Months 1–6)

- Add UWB ranging layer: replace RSSI ranging with DW3000 UWB modules for 10–30 cm precision. The EKF architecture remains identical; only the measurement noise R changes. This is the path to publishable-grade positioning accuracy.
- Implement 3D positioning: fuse barometer altitude with the 2D EKF to produce a full 3D position estimate. Add 3D trilateration solver (requires ≥4 non-coplanar anchors).
- Distributed mesh positioning: remove the ground-station bottleneck by implementing fully peer-to-peer positioning. Each drone estimates all other drones' positions from its own RSSI measurements. This requires a distributed EKF or particle filter.
- Adaptive Kalman tuning: implement Adaptive KF (AKF) that automatically adjusts Q and R based on innovation consistency tests. This eliminates manual tuning across different environments.

20.3 Long-Term Research Directions

- Machine-learning RSSI-to-distance model: replace the parametric log-distance model with a neural network trained on calibration data per environment type. Potentially reduces σ by 30–50% indoors by learning multipath patterns.
- Formation flying with position-aware feedback: implement swarm formation controller that uses EKF position estimates to compute formation error and generate corrective velocity commands for follower drones.
- Scalability to 20+ nodes: characterise TDMA bandwidth ceiling, packet loss probability, and EKF computational cost as node count scales. Transition to distributed sub-swarm architectures if required.
- Robustness testing: intentional jamming simulation (RF interference generator), packet drop stress tests, battery depletion effects on radio performance, and formation coherence during GPS signal restoration.

21. References

Hardware and Chipset Documentation

- Ebyte E22-900T22S Product Page and Datasheet (LCSC). https://www.ebyte.com/en/product-view-news.html
- Semtech SX1262 Datasheet. Rev 2.1. Semtech Corporation, 2019. https://semtech.com/products/wireless-rf/lora-transceivers/sx1262
- Semtech SX1276/77/78/79 Datasheet (LoRa RSSI accuracy reference, SX1276 family). Semtech Corporation.

Software Libraries

- RadioLib — Universal wireless communication library, SX126x support. https://github.com/jgromes/RadioLib
- ArduPilot Autopilot Firmware. https://ardupilot.org
- Meshtastic — Open-source LoRa mesh networking platform. https://meshtastic.org
- PlatformIO — Embedded development platform. https://platformio.org

Signal Processing and Positioning

- Rappaport, T.S. (2002). Wireless Communications: Principles and Practice (2nd ed.). Prentice Hall. [Path-loss model, Chapter 3]
- Harle, R. (2013). A Survey of Indoor Inertial Positioning Systems for Pedestrians. IEEE Communications Surveys & Tutorials.
- Welch, G., Bishop, G. (2006). An Introduction to the Kalman Filter. UNC Chapel Hill Technical Report TR 95-041.
- Thrun, S., Burgard, W., Fox, D. (2005). Probabilistic Robotics. MIT Press. [EKF and particle filter for robot positioning]

Project Repository

- EDTH-LORA GitHub Repository: Challenge #13 · LoRa Swarm Drone · AirQit / Mike Dewhirst · EDTH Hackathon 2026
- Simulation videos: https://drive.google.com/drive/folders/132m6eond6za0xi-JsWrmCsyZc5dGiNe_